ClaimCheck v0.1 — "Talking Your Book" Detector

> Traces public claims through conflict-of-interest causal chains.
> Architecture inspired by FinRAG-OS effect propagation.

Run order: Cell 1 → 2 → 3 → 4 → 5 → 6

CELL 1 — Install + Config + LLM Wrapper

```
import os, json, time, re, textwrap
from dataclasses import dataclass, field
from typing import List, Dict, Optional, Any, Tuple

# ═══════════════════════════════════════════════════════════════
# MODEL CONFIG — Same tiering pattern as FinRAG-OS
# ═══════════════════════════════════════════════════════════════
MODEL_TIER = "BUDGET"  # ← CHANGE THIS to switch models

MODEL_OPTIONS = {
    "FREE": {
        "model_id": "deepseek/deepseek-chat:free",
        "name": "DeepSeek V3 (Free)", "input_cost": 0.00, "output_cost": 0.00,
        "note": "Rate-limited. Smoke tests only.",
    },
    "BUDGET": {
        "model_id": "deepseek/deepseek-chat",
        "name": "DeepSeek V3", "input_cost": 0.32, "output_cost": 0.89,
        "note": "Best value. ~$0.003/claim analysis.",
    },
    "QUALITY": {
        "model_id": "deepseek/deepseek-v3.2",
        "name": "DeepSeek V3.2", "input_cost": 0.26, "output_cost": 0.38,
        "note": "Near-frontier. Best quality-per-dollar.",
    },
    "PREMIUM": {
        "model_id": "meta-llama/llama-4-scout",
        "name": "Llama 4 Scout", "input_cost": 0.15, "output_cost": 0.60,
        "note": "Strong reasoning. 50x more expensive than DeepSeek.",
    },
    "FRONTIER": {
        "model_id": "anthropic/claude-sonnet-4",
        "name": "Claude Sonnet 4", "input_cost": 3.00, "output_cost": 15.00,
        "note": "Best quality. Use for final eval only.",
    },
}

selected = MODEL_OPTIONS[MODEL_TIER]
print(f"🤖  Model: {selected['name']} (tier: {MODEL_TIER})")
print(f"   Cost: ${selected['input_cost']:.2f}/${selected['output_cost']:.2f} per
```

```
print(f"   Note: {selected['note']}")

# ═══════════════════════════════════════════════════════════════════
# OpenRouter LLM Wrapper (from FinRAG-OS)
# ═══════════════════════════════════════════════════════════════════
import requests as req_lib

try:
    from google.colab import userdata
    COLAB_ENV = True
except ImportError:
    COLAB_ENV = False

class OpenRouterLLM:
    def __init__(self, model_id: str):
        self.model_id = model_id
        self.url = "https://openrouter.ai/api/v1/chat/completions"
        self.total_calls = 0
        self.total_input_tokens = 0
        self.total_output_tokens = 0

        if COLAB_ENV:
            raw_key = userdata.get("OPENROUTER_API_KEY")
            self.api_key = raw_key.strip() if raw_key else None
        else:
            self.api_key = os.environ.get("OPENROUTER_API_KEY")

        if not self.api_key:
            print("❌  OPENROUTER_API_KEY not set!")
            return

        # Early validation
        print(f"🔑  Validating API key for {model_id}...", end="")
        headers = {"Authorization": f"Bearer {self.api_key}",
                   "Content-Type": "application/json",
                   "HTTP-Referer": "https://github.com/claimcheck"}
        payload = {"model": self.model_id,
                   "messages": [{"role": "user", "content": "Say OK"}],
                   "max_tokens": 5, "temperature": 0.0}
        try:
            r = req_lib.post(self.url, headers=headers, json=payload, timeout=30)
            if r.status_code == 200:
                print(" ✅  Key valid!")
            else:
                print(f"\n❌  HTTP {r.status_code}: {r.text[:150]}")
        except Exception as e:
            print(f"\n⚠️  Validation failed: {e}")

    def generate(self, prompt: str, max_tokens: int = 600, temperature: float = 0.
                 system_prompt: str = None) -> str:
        if not self.api_key:
            return "Error: API key not set"
        messages = []
        if system_prompt:
            messages.append({"role": "system", "content": system_prompt})
        messages.append({"role": "user", "content": prompt})
        headers = {"Authorization": f"Bearer {self.api_key}",
```

```
                    "Content-Type": "application/json",
                    "HTTP-Referer": "https://github.com/claimcheck"}
        payload = {"model": self.model_id, "messages": messages,
                    "max_tokens": max_tokens, "temperature": temperature}
        for attempt in range(3):
            try:
                r = req_lib.post(self.url, headers=headers, json=payload, timeout=
                if r.status_code == 200:
                    data = r.json()
                    self.total_calls += 1
                    text = data["choices"][0]["message"]["content"].strip()
                    usage = data.get("usage", {})
                    self.total_input_tokens += usage.get("prompt_tokens", 0)
                    self.total_output_tokens += usage.get("completion_tokens", 0)
                    return text
                elif r.status_code == 429:
                    time.sleep(2 ** attempt)
                    continue
                else:
                    return f"Error: HTTP {r.status_code}: {r.text[:200]}"
            except Exception as e:
                if attempt < 2:
                    time.sleep(2)
                    continue
                return f"Error: {e}"
        return "Error: Max retries exceeded"

    def cost_so_far(self) -> float:
        m = selected
        return (self.total_input_tokens * m["input_cost"] / 1_000_000 +
                self.total_output_tokens * m["output_cost"] / 1_000_000)

    def stats(self) -> str:
        return (f"📊 LLM Stats: {self.total_calls} calls | "
                f"{self.total_input_tokens:,} in / {self.total_output_tokens:,} ou
                f"Cost: ${self.cost_so_far():.4f}")

llm = OpenRouterLLM(selected["model_id"])
```

```
🤖 Model: DeepSeek V3 (tier: BUDGET)
   Cost: $0.32/$0.89 per 1M tokens
   Note: Best value. ~$0.003/claim analysis.
🔑 Validating API key for deepseek/deepseek-chat... ✅ Key valid!
```

CELL 2 — Person Database (Tier 1: Hand-Curated)

```
# ═══════════════════════════════════════════════════════════════════
# TIER 1: Rigorously researched financial ties for high-profile figures
# This is the competitive moat. Each entry is hand-verified.
# ═══════════════════════════════════════════════════════════════════

PERSON_DB = {
    "jensen_huang": {
```

```
            "name": "Jensen Huang",
            "title": "CEO & Co-founder, NVIDIA",
            "primary_company": "NVIDIA",
            "primary_ticker": "NVDA",
            "financial_ties": [
                {"type": "equity", "entity": "NVIDIA (NVDA)", "detail": "Owns ~3.5% o
                {"type": "compensation", "entity": "NVIDIA", "detail": "CEO comp tie
                {"type": "supply_chain", "entity": "TSMC (TSM)", "detail": "NVIDIA is
            ],
            "benefits_when": [
                "gpu demand increases", "ai compute spending rises", "data center cap
                "ai training demand increases", "ai inference demand increases",
                "cloud providers buy more gpus", "autonomous vehicle compute demand
            ],
        },
        "elon_musk": {
            "name": "Elon Musk",
            "title": "CEO of Tesla & SpaceX; Founder of xAI; Owner of X",
            "primary_company": "Tesla",
            "primary_ticker": "TSLA",
            "financial_ties": [
                {"type": "equity", "entity": "Tesla (TSLA)", "detail": "Owns ~13% of
                {"type": "founder", "entity": "xAI (Grok)", "detail": "Founded xAI,
                {"type": "owner", "entity": "X (Twitter)", "detail": "Acquired for $
                {"type": "founder", "entity": "SpaceX", "detail": "~42% ownership, ~
                {"type": "founder", "entity": "Neuralink", "detail": "Brain-computer
                {"type": "founder", "entity": "The Boring Company", "detail": "Tunne
                {"type": "political", "entity": "US Gov (DOGE)", "detail": "Leads Dep
            ],
            "benefits_when": [
                "ev adoption accelerates", "ai hype increases", "autonomous driving
                "twitter/x engagement grows", "ai compute demand rises", "government
                "crypto prices rise", "robotics hype increases", "space industry grow
            ],
        },
        "sam_altman": {
            "name": "Sam Altman",
            "title": "CEO of OpenAI",
            "primary_company": "OpenAI",
            "primary_ticker": None,
            "financial_ties": [
                {"type": "equity", "entity": "OpenAI", "detail": "CEO with ~$10B+ equ
                {"type": "partnership", "entity": "Microsoft (MSFT)", "detail": "MSFT
                {"type": "investment", "entity": "Helion Energy", "detail": "Persona
                {"type": "founder", "entity": "Worldcoin/World", "detail": "Co-founde
            ],
            "benefits_when": [
                "ai adoption increases", "agi narrative gains traction", "enterprise
                "openai api usage grows", "chatgpt subscriptions increase", "microso
                "ai regulation favors incumbents", "nuclear/fusion energy demand grow
            ],
        },
        "chamath_palihapitiya": {
            "name": "Chamath Palihapitiya",
            "title": "CEO of Social Capital; All-In Podcast host",
            "primary_company": "Social Capital",
```

```
            "primary_ticker": None,
            "financial_ties": [
                {"type": "fund", "entity": "Social Capital", "detail": "Manages tech,
                {"type": "spac", "entity": "Various SPACs (IPOA-F)", "detail": "Prom
                {"type": "media", "entity": "All-In Podcast", "detail": "Co-host; na
                {"type": "investment", "entity": "Crypto holdings", "detail": "Known
            ],
            "benefits_when": [
                "tech sector rallies", "crypto prices rise", "spac market recovers",
                "healthcare innovation hype", "climate tech gets funding", "podcast a
                "anti-establishment finance narrative gains traction",
            ],
        },
        "marc_andreessen": {
            "name": "Marc Andreessen",
            "title": "Co-founder & GP, a16z (Andreessen Horowitz)",
            "primary_company": "a16z",
            "primary_ticker": None,
            "financial_ties": [
                {"type": "fund", "entity": "a16z", "detail": "Co-founded a16z (~$42B
                {"type": "board", "entity": "Meta (META)", "detail": "Board member o
                {"type": "portfolio", "entity": "AI startups (Mistral, Character.ai,
                {"type": "portfolio", "entity": "Crypto/Web3", "detail": "a16z crypt
            ],
            "benefits_when": [
                "ai startup valuations rise", "crypto adoption increases", "tech der
                "meta stock appreciates", "enterprise ai adoption grows", "open sour
                "anti-regulation sentiment grows",
            ],
        },
        "satya_nadella": {
            "name": "Satya Nadella",
            "title": "CEO of Microsoft",
            "primary_company": "Microsoft",
            "primary_ticker": "MSFT",
            "financial_ties": [
                {"type": "equity", "entity": "Microsoft (MSFT)", "detail": "~$50M+ a
                {"type": "partnership", "entity": "OpenAI", "detail": "MSFT invested
            ],
            "benefits_when": [
                "enterprise ai adoption grows", "azure cloud grows", "copilot subscr
                "ai pc refresh cycle starts", "openai succeeds", "github copilot ado
            ],
        },
        "sundar_pichai": {
            "name": "Sundar Pichai",
            "title": "CEO of Alphabet/Google",
            "primary_company": "Alphabet",
            "primary_ticker": "GOOGL",
            "financial_ties": [
                {"type": "equity", "entity": "Alphabet (GOOGL)", "detail": "~$200M+ a
            ],
            "benefits_when": [
                "ai search adoption grows", "google cloud grows", "ai integration dr
                "youtube engagement grows", "android ecosystem expands",
            ],
```

```
    },
    "lisa_su": {
        "name": "Lisa Su",
        "title": "CEO of AMD",
        "primary_company": "AMD",
        "primary_ticker": "AMD",
        "financial_ties": [
            {"type": "equity", "entity": "AMD (AMD)", "detail": "CEO comp heavily
        ],
        "benefits_when": [
            "ai chip demand grows", "data center gpu demand rises", "cpu market s
            "ai inference demand increases",
        ],
    },
    "david_sacks": {
        "name": "David Sacks",
        "title": "GP at Craft Ventures; White House AI & Crypto Czar",
        "primary_company": "Craft Ventures",
        "primary_ticker": None,
        "financial_ties": [
            {"type": "fund", "entity": "Craft Ventures", "detail": "GP, invested
            {"type": "political", "entity": "US Gov (AI & Crypto Czar)", "detail"
            {"type": "portfolio", "entity": "Crypto projects", "detail": "Heavy c
        ],
        "benefits_when": [
            "crypto deregulation happens", "ai deregulation happens", "enterprise
            "pro-crypto policy enacted", "tech-friendly regulation passes",
        ],
    },
}

print(f"✅  Loaded {len(PERSON_DB)} Tier 1 profiles")
for pid, p in PERSON_DB.items():
    print(f"   • {p['name']} — {len(p['financial_ties'])} ties, {len(p['benefits_
```

```
✅  Loaded 9 Tier 1 profiles
   • Jensen Huang — 3 ties, 7 benefit triggers
   • Elon Musk — 7 ties, 9 benefit triggers
   • Sam Altman — 4 ties, 8 benefit triggers
   • Chamath Palihapitiya — 4 ties, 7 benefit triggers
   • Marc Andreessen — 4 ties, 7 benefit triggers
   • Satya Nadella — 2 ties, 6 benefit triggers
   • Sundar Pichai — 1 ties, 5 benefit triggers
   • Lisa Su — 1 ties, 4 benefit triggers
   • David Sacks — 3 ties, 5 benefit triggers
```

CELL 3 — Core Analysis: Multi-Hypothesis Conflict Chain

```
# ═══════════════════════════════════════════════════════════════════
# STEP A: Claim Decomposition
# ═══════════════════════════════════════════════════════════════════

DECOMPOSE_PROMPT = """You are a financial conflict-of-interest analyst. Decompose
```

```
    CLAIM: "{claim}"
    SPEAKER: {speaker_name} ({speaker_title})

    Extract:
    1. IMPLIED_ACTION: What is the speaker urging people/companies to DO? (one senter
    2. DOMAIN: What economic domain does this affect? (e.g., "AI compute spending", '
    3. EXTREMITY: Rate 1-10 how extreme/extraordinary this claim is (1=mild, 10=outra
    4. EXTREMITY_REASON: One sentence why you gave that rating

    Respond EXACTLY in this format (no extra text):
    IMPLIED_ACTION: ...
    DOMAIN: ...
    EXTREMITY: ...
    EXTREMITY_REASON: ..."""


    def decompose_claim(claim_text: str, speaker_id: str) -> dict:
        """Step A: Break a claim into structured components."""
        person = PERSON_DB.get(speaker_id, {})
        prompt = DECOMPOSE_PROMPT.format(
            claim=claim_text,
            speaker_name=person.get("name", speaker_id),
            speaker_title=person.get("title", "Unknown"),
        )
        raw = llm.generate(prompt, max_tokens=200, temperature=0.1)
        # Parse
        result = {"implied_action": "", "domain": "", "extremity": 5, "extremity_rea
        for line in raw.split("\n"):
            line = line.strip()
            if line.startswith("IMPLIED_ACTION:"):
                result["implied_action"] = line.split(":", 1)[1].strip()
            elif line.startswith("DOMAIN:"):
                result["domain"] = line.split(":", 1)[1].strip()
            elif line.startswith("EXTREMITY:") and not line.startswith("EXTREMITY_RE
                try:
                    result["extremity"] = int(re.search(r'\d+', line.split(":", 1)[1
                except:
                    pass
            elif line.startswith("EXTREMITY_REASON:"):
                result["extremity_reason"] = line.split(":", 1)[1].strip()
        return result


    # ═══════════════════════════════════════════════════════════════
    # STEP B: Multi-Hypothesis Demand Generation
    # ═══════════════════════════════════════════════════════════════

    HYPOTHESIS_PROMPT = """You are tracing the economic supply chain downstream from

    CLAIM: "{claim}"
    IMPLIED ACTION: "{implied_action}"
    DOMAIN: "{domain}"

    Generate exactly 5 HYPOTHESES for who economically benefits if people follow this
    For each, trace the chain: behavior change → intermediate demand → specific compa
```

```
Be specific. Name actual companies where possible (e.g., "NVIDIA", "AWS", "OpenA
Think about: direct suppliers, infrastructure providers, platform owners, adjace

Format EXACTLY (one line each, no extra text):
H1: [behavior] → [intermediate] → [beneficiary companies/sectors]
H2: ...
H3: ...
H4: ...
H5: ..."""


def generate_hypotheses(claim_text: str, decomposition: dict) -> List[dict]:
    """Step B: Generate 5 hypotheses for who benefits."""
    prompt = HYPOTHESIS_PROMPT.format(
        claim=claim_text,
        implied_action=decomposition["implied_action"],
        domain=decomposition["domain"],
    )
    raw = llm.generate(prompt, max_tokens=500, temperature=0.3)
    hypotheses = []
    for line in raw.split("\n"):
        line = line.strip()
        match = re.match(r'H(\d):\s*(.*)', line)
        if match:
            hypotheses.append({
                "id": f"H{match.group(1)}",
                "chain": match.group(2).strip(),
            })
    return hypotheses


# ═══════════════════════════════════════════════════════════════════
# STEP C: Conflict Matching
# ═══════════════════════════════════════════════════════════════════

MATCH_PROMPT = """You are scoring how self-serving a public claim is.

CLAIM: "{claim}"
SPEAKER: {speaker_name} ({speaker_title})

SPEAKER'S KNOWN FINANCIAL TIES:
{ties_text}

SPEAKER BENEFITS WHEN:
{benefits_text}

HYPOTHESIZED BENEFICIARY CHAINS:
{hypotheses_text}

For EACH hypothesis, determine:
- Does the beneficiary match ANY of the speaker's financial ties or benefit trig
- How direct is the connection? (DIRECT/INDIRECT/NONE)
- Confidence? (HIGH/MEDIUM/LOW)

Then provide:
```

```
- DOUBT_REASONS: 2-3 specific, concrete reasons this claim seems self-serving
- FAIR_POINTS: 1-2 reasons the claim might still be genuine
- BS_SUMMARY: One sentence verdict

Format EXACTLY:
H1: [DIRECT/INDIRECT/NONE] | [HIGH/MEDIUM/LOW] | [brief explanation]
H2: ...
H3: ...
H4: ...
H5: ...
DOUBT_REASONS:
1. ...
2. ...
3. ...
FAIR_POINTS:
1. ...
2. ...
BS_SUMMARY: ..."""


def match_conflicts(claim_text: str, speaker_id: str, hypotheses: List[dict]) ->
    """Step C: Match hypotheses against speaker's financial ties."""
    person = PERSON_DB.get(speaker_id, {})

    ties_text = "\n".join(f"  - [{t['type']}] {t['entity']}: {t['detail']}" for t
    benefits_text = "\n".join(f"  - {b}" for b in person.get("benefits_when", []
    hyp_text = "\n".join(f"  {h['id']}: {h['chain']}" for h in hypotheses)

    prompt = MATCH_PROMPT.format(
        claim=claim_text,
        speaker_name=person.get("name", speaker_id),
        speaker_title=person.get("title", "Unknown"),
        ties_text=ties_text or "  (none in database)",
        benefits_text=benefits_text or "  (none in database)",
        hypotheses_text=hyp_text,
    )
    raw = llm.generate(prompt, max_tokens=700, temperature=0.1)

    # Parse matches
    matches = []
    doubt_reasons = []
    fair_points = []
    bs_summary = ""
    section = None

    for line in raw.split("\n"):
        line = line.strip()
        h_match = re.match(r'H(\d):\s*(DIRECT|INDIRECT|NONE)\s*\|\s*(HIGH|MEDIUM
        if h_match:
            matches.append({
                "hypothesis": f"H{h_match.group(1)}",
                "connection": h_match.group(2),
                "confidence": h_match.group(3),
                "explanation": h_match.group(4).strip(),
            })
            continue
```

```
        if line.startswith("DOUBT_REASONS"):
            section = "doubt"
            continue
        elif line.startswith("FAIR_POINTS"):
            section = "fair"
            continue
        elif line.startswith("BS_SUMMARY:"):
            bs_summary = line.split(":", 1)[1].strip()
            section = None
            continue
        if section == "doubt" and re.match(r'\d+\.', line):
            doubt_reasons.append(re.sub(r'^\d+\.\s*', '', line))
        elif section == "fair" and re.match(r'\d+\.', line):
            fair_points.append(re.sub(r'^\d+\.\s*', '', line))

    return {
        "matches": matches,
        "doubt_reasons": doubt_reasons,
        "fair_points": fair_points,
        "bs_summary": bs_summary,
        "raw": raw,
    }


print("✅  Core analysis functions defined (decompose → hypothesize → match)")
```

```
✅  Core analysis functions defined (decompose → hypothesize → match)
```

CELL 4 — BS Score Calculator + Report Generator

```
def calculate_bs_score(matches: List[dict], extremity: int, person: dict) -> dic
    """
    Calculate BS Percentile (0-100) from conflict analysis.

    Components:
    1. Connection Rate (0-40): What % of hypotheses connect back?
    2. Directness (0-30): How direct are the strongest connections?
    3. Financial Magnitude (0-20): How much money is at stake?
    4. Claim Extremity (0-10): How exaggerated is the claim?
    """
    if not matches:
        return {"score": 0, "connection_rate": 0, "directness": 0, "magnitude": (

    # 1. Connection Rate (0-40)
    connected = [m for m in matches if m["connection"] != "NONE"]
    connection_rate = (len(connected) / max(len(matches), 1)) * 40

    # 2. Directness (0-30)
    directness_map = {"DIRECT": 1.0, "INDIRECT": 0.5, "NONE": 0.0}
    confidence_map = {"HIGH": 1.0, "MEDIUM": 0.7, "LOW": 0.4}
    if connected:
        max_directness = max(
            directness_map.get(m["connection"], 0) * confidence_map.get(m["confi
```

```
                for m in connected
            )
        else:
            max_directness = 0
        directness_score = max_directness * 30

        # 3. Financial Magnitude (0-20) - proxy from number of direct ties
        direct_ties = [t for t in person.get("financial_ties", []) if t.get("directn
        magnitude_score = min(len(direct_ties) / 3.0, 1.0) * 20  # caps at 3+ direct

        # 4. Extremity (0-10)
        extremity_score = (min(extremity, 10) / 10.0) * 10

        total = connection_rate + directness_score + magnitude_score + extremity_sco
        return {
            "score": round(min(total, 100)),
            "connection_rate": round(connection_rate, 1),
            "directness": round(directness_score, 1),
            "magnitude": round(magnitude_score, 1),
            "extremity_score": round(extremity_score, 1),
        }


    def score_emoji(score: int) -> str:
        if score >= 80: return "🔴"
        elif score >= 60: return "🟠"
        elif score >= 40: return "🟡"
        elif score >= 20: return "🟢"
        else: return "⚪"


    def format_report(claim_text: str, speaker_id: str, decomposition: dict,
                      hypotheses: List[dict], conflict: dict, bs: dict) -> str:
        """Format the full analysis report for one claim."""
        person = PERSON_DB.get(speaker_id, {})
        emoji = score_emoji(bs["score"])
        ticker_str = f" | {person['primary_ticker']}" if person.get("primary_ticker"

        lines = []
        lines.append("=" * 70)
        lines.append(f'📢 CLAIM: "{claim_text}"')
        lines.append(f"🗣️  SPEAKER: {person.get('name', speaker_id)} ({person.get('t
        lines.append("")
        lines.append(f"{emoji} BS PERCENTILE: {bs['score']}/100")
        lines.append(f"   Connection: {bs['connection_rate']}/40 | Directness: {bs['
                     f"Magnitude: {bs['magnitude']}/20 | Extremity: {bs['extremity_s
        lines.append("")

        # Decomposition
        lines.append(f"📋 IMPLIED ACTION: {decomposition.get('implied_action', 'N/A'
        lines.append(f"   Domain: {decomposition.get('domain', 'N/A')}")
        lines.append(f"   Extremity: {decomposition.get('extremity', '?')}/10 - {dec
        lines.append("")

        # Conflict chains
        lines.append("🔗 CONFLICT CHAINS:")
```

```
    for m in conflict.get("matches", []):
        icon = "✅" if m["connection"] == "DIRECT" else ("⚠️" if m["connection"]
        h = next((h for h in hypotheses if h["id"] == m["hypothesis"]), None)
        chain_text = h["chain"] if h else "?"
        lines.append(f"   {icon} {m['hypothesis']}: {chain_text}")
        lines.append(f"      → {m['connection']} | Confidence: {m['confidence']}
    lines.append("")

    # Doubt reasons
    if conflict.get("doubt_reasons"):
        lines.append("🤔 WHAT MADE US DOUBT:")
        for i, r in enumerate(conflict["doubt_reasons"], 1):
            lines.append(f"   {i}. {r}")
        lines.append("")

    # Fair points
    if conflict.get("fair_points"):
        lines.append("⚖️  TO BE FAIR:")
        for i, r in enumerate(conflict["fair_points"], 1):
            lines.append(f"   {i}. {r}")
        lines.append("")

    # Summary
    if conflict.get("bs_summary"):
        lines.append(f"📝 VERDICT: {conflict['bs_summary']}")

    lines.append("=" * 70)
    return "\n".join(lines)


print("✅  BS Score calculator + report generator defined")
```

```
✅  BS Score calculator + report generator defined
```

CELL 5 — Pipeline Orchestrator

```
def analyze_claim(claim_text: str, speaker_id: str, verbose: bool = True) -> dic
    """
    Full pipeline: Claim → Decompose → Hypothesize → Match → Score → Report

    Returns dict with all intermediate results for inspection.
    """
    person = PERSON_DB.get(speaker_id)
    if not person:
        print(f"⚠️  Speaker '{speaker_id}' not in Tier 1 DB. Using agent fallbac
        return {"error": f"Speaker {speaker_id} not found. Add to PERSON_DB or i

    if verbose:
        print(f"\n{'-' * 60}")
        print(f"Analyzing: {person['name']}")
        print(f'Claim: "{claim_text[:80]}..."' if len(claim_text) > 80 else f'Cl

    # Step A: Decompose
```

```
    if verbose: print("  [1/4] Decomposing claim...")
    decomposition = decompose_claim(claim_text, speaker_id)
    if verbose: print(f"         → Action: {decomposition['implied_action'][:60]

    # Step B: Generate hypotheses
    if verbose: print("  [2/4] Generating hypotheses...")
    hypotheses = generate_hypotheses(claim_text, decomposition)
    if verbose: print(f"         → {len(hypotheses)} hypotheses generated")

    # Step C: Match conflicts
    if verbose: print("  [3/4] Matching against financial ties...")
    conflict = match_conflicts(claim_text, speaker_id, hypotheses)
    connected = [m for m in conflict["matches"] if m["connection"] != "NONE"]
    if verbose: print(f"         → {len(connected)}/{len(conflict['matches'])} hy

    # Step D: Calculate score
    if verbose: print("  [4/4] Calculating BS score...")
    bs = calculate_bs_score(conflict["matches"], decomposition["extremity"], pers
    if verbose: print(f"         → BS Percentile: {bs['score']}/100 {score_emoji

    # Format report
    report = format_report(claim_text, speaker_id, decomposition, hypotheses, con

    return {
        "claim": claim_text,
        "speaker_id": speaker_id,
        "decomposition": decomposition,
        "hypotheses": hypotheses,
        "conflict": conflict,
        "bs_score": bs,
        "report": report,
    }


print("✅  Pipeline orchestrator defined. Ready to analyze claims.")
```

```
✅  Pipeline orchestrator defined. Ready to analyze claims.
```

CELL 6 — Run Analysis on Sample Claims

```
# ═══════════════════════════════════════════════════════════════
# SAMPLE CLAIMS — Edit these or add your own
# ═══════════════════════════════════════════════════════════════

SAMPLE_CLAIMS = [
    {
        "claim": "Every employee paid $500K should be burning $250K worth of AI t
        "speaker": "jensen_huang",
    },
    {
        "claim": "AI will be smarter than any single human by end of next year."
        "speaker": "elon_musk",
    },
```

```
    {
        "claim": "We are beginning to see the early glimpses of AGI. GPT-4 was tl
        "speaker": "sam_altman",
    },
    {
        "claim": "Every company needs to become an AI company or they will be di:
        "speaker": "satya_nadella",
    },
    {
        "claim": "Bitcoin is going to $500,000. It's the hardest money ever crea
        "speaker": "chamath_palihapitiya",
    },
]

# ═══════════════════════════════════════════════════════════════════════
# RUN PIPELINE
# ═══════════════════════════════════════════════════════════════════════

results = []
print("🚀 ClaimCheck v0.1 — Analyzing claims...\n")

for i, item in enumerate(SAMPLE_CLAIMS):
    print(f"\n{'▓' * 70}")
    print(f"  CLAIM {i+1}/{len(SAMPLE_CLAIMS)}")
    print(f"{'▓' * 70}")

    result = analyze_claim(item["claim"], item["speaker"])
    results.append(result)

    if "report" in result:
        print(f"\n{result['report']}")
    print()

# ═══════════════════════════════════════════════════════════════════════
# SUMMARY TABLE
# ═══════════════════════════════════════════════════════════════════════
print("\n\n" + "▇" * 70)
print("  📊 SUMMARY — BS LEADERBOARD")
print("▇" * 70)

sorted_results = sorted([r for r in results if "bs_score" in r],
                        key=lambda x: x["bs_score"]["score"], reverse=True)

for i, r in enumerate(sorted_results, 1):
    person = PERSON_DB.get(r["speaker_id"], {})
    score = r["bs_score"]["score"]
    emoji = score_emoji(score)
    claim_short = r["claim"][:50] + "..." if len(r["claim"]) > 50 else r["claim"]
    print(f"  {i}. {emoji} {score:3d}/100  {person.get('name', '?'):20s}  \"{cla:

print(f"\n{llm.stats()}")
print(f"{'▇' * 70}")
```

```
[2/4] Generating hypotheses...
      → 5 hypotheses generated
[3/4] Matching against financial ties...
      → 5/5 hypotheses connect back
[4/4] Calculating BS score...
      → BS Percentile: 80/100 🔴


═══════════════════════════════════════════════════════════════════════
📢 CLAIM: "Bitcoin is going to $500,000. It's the hardest money ever created by
🗣️  SPEAKER: Chamath Palihapitiya (CEO of Social Capital; All-In Podcast host)

🔴 BS PERCENTILE: 80/100
   Connection: 40.0/40 | Directness: 10.5/30 | Magnitude: 20.0/20 | Extremity: 9

📋 IMPLIED ACTION: Invest in Bitcoin as it will appreciate significantly.
   Domain: crypto adoption
   Extremity: 9/10 — The claim predicts an extremely high and specific price tar

🔗 CONFLICT CHAINS:
   ⚠️ H1: Increased Bitcoin investment → higher demand for crypto exchanges → Co
      → INDIRECT | Confidence: MEDIUM | Chamath has crypto holdings, but the dir
   ⚠️ H2: Increased Bitcoin mining → higher demand for mining hardware → NVIDIA,
      → INDIRECT | Confidence: LOW | Mining hardware companies are not directly
   ⚠️ H3: Increased Bitcoin transactions → higher demand for blockchain infrastr
      → INDIRECT | Confidence: LOW | Blockchain infrastructure companies are not
   ⚠️ H4: Increased Bitcoin adoption → higher demand for crypto wallets → Ledger
      → INDIRECT | Confidence: LOW | Crypto wallet companies are not directly ti
   ⚠️ H5: Increased Bitcoin speculation → higher demand for financial news platf
      → INDIRECT | Confidence: LOW | Financial news platforms are not directly t

🤔 WHAT MADE US DOUBT:
   1. Chamath has known Bitcoin/crypto positions, so promoting Bitcoin's rise di
   2. His podcast and media presence amplify narratives that align with his inve
   3. His history of promoting speculative assets (e.g., SPACs) suggests a patte

⚖️  TO BE FAIR:
   1. Bitcoin's potential as "hard money" is a debated topic in finance, and Cha
   2. His claim aligns with broader bullish sentiment in the crypto community, n

📝 VERDICT: Chamath's claim appears self-serving given his crypto holdings and h
═══════════════════════════════════════════════════════════════════════



═══════════════════════════════════════════════════════════════════════
📊 SUMMARY — BS LEADERBOARD
═══════════════════════════════════════════════════════════════════════
  1. 🔴  91/100  Jensen Huang          "Every employee paid $500K should be burn
  2. 🔴  91/100  Elon Musk             "AI will be smarter than any single human
  3. 🔴  90/100  Sam Altman            "We are beginning to see the early glimps
  4. 🔴  85/100  Satya Nadella         "Every company needs to become an AI comp
  5. 🔴  80/100  Chamath Palihapitiya  "Bitcoin is going to $500,000. It's the h

📊 LLM Stats: 15 calls | 4,593 in / 2,516 out | Cost: $0.0037
═══════════════════════════════════════════════════════════════════════
```

API

```
!pip install -q flask flask-cors pyngrok
```

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import threading
import time
from pyngrok import ngrok
from werkzeug.serving import make_server

app = Flask(__name__)

# Allow your Vercel frontend to talk to this Colab backend
CORS(
    app,
    resources={r"/api/*": {"origins": ["https://bs-detector-eta.vercel.app", "http
    supports_credentials=True,
    allow_headers=["Content-Type", "ngrok-skip-browser-warning"],
    methods=["GET", "POST", "OPTIONS"],
)

# ═══════════════════════════════════════════════════════════════════════
# ENDPOINT: POST /api/analyze
# ═══════════════════════════════════════════════════════════════════════
@app.route('/api/analyze', methods=['POST'])
def analyze_claim():
    data = request.json or {}
    claim_text = data.get("claim_text")
    speaker_id = data.get("speaker_id")

    if not claim_text or not speaker_id:
        return jsonify({"error": "Missing claim_text or speaker_id"}), 400

    if speaker_id not in PERSON_DB:
        return jsonify({"error": f"Speaker {speaker_id} not found in database."}),

    speaker = PERSON_DB[speaker_id]

    try:
        # Step 1: Decompose the claim
        decomposed = decompose_claim(claim_text, speaker, OpenRouterLLM)

        # Step 2: Generate Hypotheses
        hypotheses = generate_hypotheses(decomposed, speaker, OpenRouterLLM)

        # Step 3: Match Conflicts
        matched = match_conflicts(hypotheses, speaker, OpenRouterLLM)

        # Step 4: Calculate Score & Generate Output
        score_data = calculate_bs_score(matched, decomposed)

        doubt_reasons = score_data.pop("doubt_reasons", [])
        fair_points = score_data.pop("fair_points", [])
        final_score = score_data.pop("final_score", 0)

        # Format the exact JSON your frontend expects
        result = {
            "claimQuote": claim_text,
            "speakerName": speaker["name"],
            "speakerTitle": speaker["title"],
```

```
            "speakerCompany": speaker["company"],
            "speakerTicker": speaker.get("ticker", ""),
            "finalScore": final_score,
            "components": score_data,
            "decomposition": decomposed,
            "hypotheses": matched,
            "doubtReasons": doubt_reasons,
            "fairPoints": fair_points
        }

        return jsonify(result)

    except Exception as e:
        print(f"Error during analysis: {e}")
        return jsonify({"error": str(e)}), 500


# ═══════════════════════════════════════════════════════════════════
# ENDPOINT: GET /api/speakers
# ═══════════════════════════════════════════════════════════════════
@app.route('/api/speakers', methods=['GET'])
def get_speakers():
    """Returns the list of speakers so the frontend dropdown can stay synced."""
    speakers_list = []
    for s_id, data in PERSON_DB.items():
        speakers_list.append({
            "id": s_id,
            "name": data["name"],
            "company": data["company"]
        })
    return jsonify(speakers_list)


# ═══════════════════════════════════════════════════════════════════
# START NGROK & FLASK SERVER
# ═══════════════════════════════════════════════════════════════════
try:
    from google.colab import userdata
    NGROK_AUTH_TOKEN = userdata.get('NGROK_AUTH_TOKEN')
except ImportError:
    import os
    NGROK_AUTH_TOKEN = os.environ.get('NGROK_AUTH_TOKEN')

if not NGROK_AUTH_TOKEN:
    print("❌ Error: NGROK_AUTH_TOKEN not found in secrets!")
else:
    print("🧹 Cleaning up existing ngrok sessions...")
    ngrok.kill()
    ngrok.set_auth_token(NGROK_AUTH_TOKEN)

    class ServerThread(threading.Thread):
        def __init__(self, app):
            threading.Thread.__init__(self)
            self.server = make_server('0.0.0.0', 5000, app)
            self.ctx = app.app_context()
            self.ctx.push()
        def run(self):
```

```
        def run(self):
            self.server.serve_forever()
        def shutdown(self):
            self.server.shutdown()

    if 'api_server_thread' in globals():
        try:
            api_server_thread.shutdown()
        except:
            pass

    api_server_thread = ServerThread(app)
    api_server_thread.start()
    print("\n🚀 Flask API server is now running in the background!")

    time.sleep(2)
    public_url = ngrok.connect(5000)

    print(f"\n✅ ClaimCheck API is live at: {public_url.public_url}")
    print(f"🔗 Update your frontend main.js to use this base URL!")
```

```
🧹 Cleaning up existing ngrok sessions...

🚀 Flask API server is now running in the background!

✅ ClaimCheck API is live at: https://ffe1-34-87-88-26.ngrok-free.app
🔗 Update your frontend main.js to use this base URL!
```